SEMAINE 1

Rappels NN & Représentations

CNN · RNN · Attention & Transformers

MISE EN CONTEXTE — DU RECONNAÎTRE AU GÉNÉRER

"Bienvenue. Ce cours s'appelle **Modèles Génératifs Profonds**. Le mot-clé, c'est **générer** : pas reconnaître, pas classifier, mais **créer du nouveau**.
Mais pour générer, il faut d'abord comprendre comment les réseaux **représentent** le monde : comment une image de chat devient un vecteur ? comment une phrase devient un point dans un espace ?
Aujourd'hui : CNN, RNN, Attention. Pas pour redire à quoi ça sert, mais pour répondre à une question : **où vit la représentation ?** Et on finira par la cassure : **reconstruire** $\neq$ **générer**." *Dans ce cours, on veut comprendre comment un modèle peut apprendre la structure des données et ensuite générer de nouvelles données. Pour ça, on a besoin de deux briques : (1) des architectures (NN, CNN, RNN, attention) et (2) des représentations (embeddings, espace latent). Aujourd'hui, on révise ces idées, mais avec un angle précis : comment on passe d'un réseau qui reconnaît à un réseau qui peut générer.*

QUESTION CENTRALE DE LA SEMAINE

Comment représenter les données de manière utile avec un réseau de neurones ?

LES TROIS QUESTIONS À POUVOIR RÉSOUDRE À LA FIN DE S1

Le cours conserve la question centrale précédente, mais l'étudiant doit être capable de la décliner en trois questions plus concrètes :

1. **Qu'est-ce qu'une donnée et comment un réseau de neurones la représente-t-il ?**
2. **Qu'est-ce qu'une représentation apprise et quel rôle joue l'espace latent ?**
3. **Que signifie concrètement reconstruire, générer, échantillonner et évaluer la plausibilité d'une donnée ?**

Ces trois questions restent volontairement centrées sur la représentation et le passage vers la génération. La comparaison formelle entre modèles discriminatifs et génératifs sera étudiée en S2.

S1 — OBJECTIFS (CE QUE VOUS SAUREZ FAIRE À LA FIN)

1. Identifier **où se trouve la représentation** dans chaque architecture (CNN, RNN, Attention/-Transformer).
2. Expliquer pourquoi un **embedding** est une **géométrie apprise**.
3. Distinguer **reconstruction** et **génération** par le rôle de l'**espace latent**.

Objectif S1 : architectures + représentations → préparation à la génération

COMPLÉMENT D'OBJECTIFS — VOCABULAIRE ESSENTIEL DE S1

En complément des trois objectifs déjà annoncés, vous devrez également pouvoir :
- distinguer X, x, x_i, $p_{\text{data}}(x)$ et $p_\theta(x)$;
- distinguer intuitivement **probabilité**, **densité** et **échantillon** ;
- expliquer ce que signifie **échantillonner** une nouvelle donnée ;
- expliquer la différence entre **représentation**, **reconstruction** et **génération** ;
- expliquer pourquoi une bonne reconstruction ne garantit pas automatiquement une bonne génération.

1 Rappels : données, variables et distributions

POURQUOI CE RAPPEL AVANT DE PARLER DE RÉSEAUX ?

Dans la suite du cours, des expressions comme $p(x)$ ou $x \sim p(x)$ apparaîtront régulièrement. Avant de demander comment un réseau construit une représentation, il faut donc préciser ce que représentent la donnée, la variable aléatoire, l'échantillon *et* la distribution.

Une observation n'est pas une distribution

NOTATION MINIMALE À CONNAÎTRE

- X : une **variable aléatoire**, c'est-à-dire l'objet abstrait qui peut prendre différentes valeurs ;
- x : une **réalisation** ou observation particulière de X ;
- x_i : une composante de l'observation x ;
- $p_{\text{data}}(x)$: la distribution réelle des données, inconnue en pratique ;
- $p_\theta(x)$: la distribution décrite ou approximée par le modèle appris, avec paramètres θ.

EXEMPLES DE DONNÉES

Une observation x peut être une image, une phrase, un signal audio, une molécule ou une série temporelle. Pour une image 28×28, x_i peut désigner l'intensité du i-ème pixel ; pour une phrase, x_i peut désigner le token à la position i.

On suppose que les observations proviennent d'un mécanisme réel que l'on résume par :

$$x \sim p_{\text{data}}(x).$$

Cette écriture signifie : « l'observation x est un échantillon provenant de la distribution des données ». Elle ne signifie pas que x est lui-même une distribution.

Variable discrète, variable continue, probabilité et densité

Une variable **discrète** prend des valeurs dans un ensemble fini ou dénombrable, par exemple une étiquette $Y \in \{0, \dots, 9\}$. Une variable **continue** prend ses valeurs dans un espace continu, par exemple un vecteur $X \in \mathbb{R}^d$.

PROBABILITÉ VERSUS DENSITÉ

Pour une variable discrète, $p(X = x)$ peut être interprété comme une probabilité ponctuelle. Pour une variable continue, $p(x)$ désigne plutôt une **densité** évaluée au point x ; la probabilité porte alors sur une région et s'obtient par intégration. Dans les deux cas, la distribution entière est normalisée.

PREMIÈRE DÉFINITION D'UN MODÈLE GÉNÉRATIF

Un modèle génératif cherche à **définir ou induire une distribution sur les données**. Schématiquement :

$$p_\theta(x) \approx p_{\text{data}}(x).$$

Une fois le modèle appris, on souhaite notamment pouvoir produire une nouvelle observation :

$$x_{\text{nouveau}} \sim p_\theta(x).$$

Exemple : le modèle observe de nombreuses images de chats, apprend les régularités de cette population puis génère une image plausible qui n'était pas dans le jeu d'entraînement.

Limite volontaire de S1 : on ne compare pas encore les différentes façons de modéliser une distribution. Cette comparaison sera le sujet de la S2.

TROIS IDÉES À DISTINGUER DÈS S1

Évaluer la plausibilité : pour une donnée x, le modèle peut parfois lui associer une quantité probabiliste telle que $p_\theta(x)$.
Échantillonner : produire une nouvelle observation selon le mécanisme appris.
Apprendre une représentation : construire un vecteur ou un tenseur z qui résume utilement x.
Pour S1, il suffit de comprendre que ces opérations sont différentes. Leur traitement selon les familles de modèles sera étudié plus tard.

2 Partie A : Les réseaux construisent des représentations internes

NN : une fonction paramétrée (base commune à CNN/RNN/Attention)

NN (BASE COMMUNE) — NOTIONS CLÉS À RETENIR

- **Un réseau = fonction paramétrée :** $\hat{y} = f_\theta(x)$.
- **Apprendre = choisir θ :** minimiser une perte $L(\hat{y}, y)$ (descente de gradient).
- **Idée centrale du cours :** le réseau ne produit pas seulement $\hat{y}$, il fabrique aussi une **représentation interne** dans ses couches cachées.
- **Représentation / latent :** on note souvent z l'activation d'une couche intermédiaire :

$$x \xrightarrow{f_{\theta_1}} z \xrightarrow{f_{\theta_2}} \hat{y}$$

- **Pont vers CNN/RNN :** ce qui change d'une architecture à l'autre, c'est **comment** on transforme l'espace d'entrée (localité, séquence, contexte).

CE QUE FABRIQUE RÉELLEMENT LE RÉSEAU

Le réseau ne fait pas seulement une prédiction.
Il construit une **représentation interne** :

$$x \to z \to \hat{y}$$

Idée clé :

- x = données brutes
- z = espace latent (représentation utile)
- $\hat{y}$ = décision finale

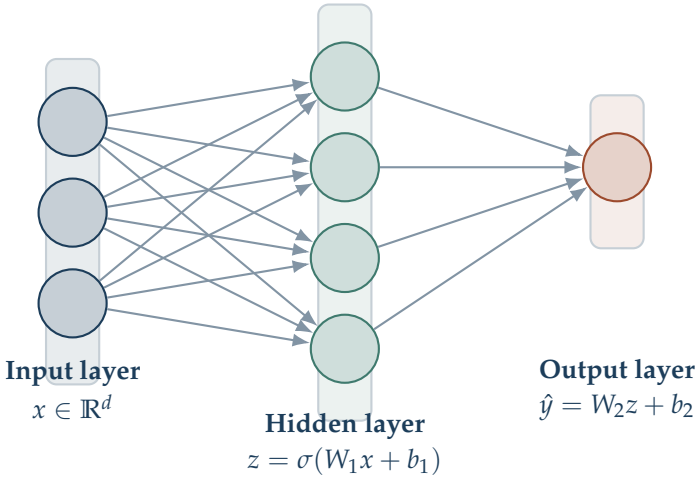
CNN, RNN et Transformers diffèrent avant tout dans la manière de construire z — même si ce choix a aussi des conséquences importantes sur la parallélisation de l'entraînement et sur la façon dont chaque architecture capture le contexte.

• APERÇU CODE — RETROUVER z DANS UN RÉSEAU

Le schéma $x \to z \to \hat{y}$ se retrouve directement dans quelques lignes de code :

```
z = torch.relu(layer1(x)) # representation interne
y_hat = layer2(z) # sortie finale
```

À observer : la variable z correspond à la représentation intermédiaire construite à partir de x ; elle n'est pas la prédiction finale.



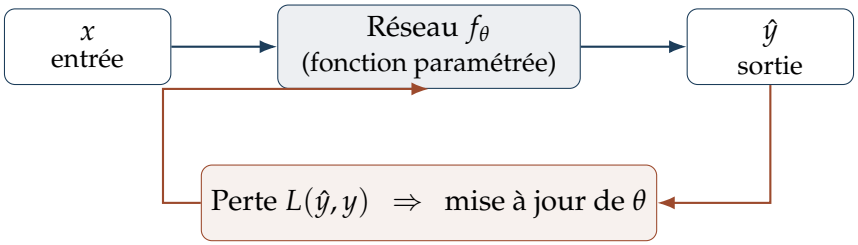
NN = CONSTRUCTEUR DE REPRÉSENTATIONS *(problème → échec → solution)*

Problème concret : on veut qu'un modèle comprenne des données complexes (images, texte, signaux) et généralise (pas juste mémoriser).
Échec (ancien outil) : travailler directement dans l'espace brut (pixels / one-hot) est fragile : les distances et variations ne reflètent pas le sens.
Solution : apprendre une transformation f_θ qui construit un espace interne (représentation) où la tâche devient plus simple.

EXPLICATION PÉDAGOGIQUE — DU RÉSEAU À LA REPRÉSENTATION

"Un réseau, c'est simplement une grande fonction composée. On choisit une architecture, puis on ajuste les paramètres θ pour réduire une erreur.
Cela vaut pour un MLP, un CNN, un RNN ou un Transformer : ce qui change, c'est la manière dont on transforme l'espace d'entrée.
Et aujourd'hui, ce qui nous intéresse surtout, ce n'est pas uniquement la sortie $\hat{y}$, mais ce qu'il se passe au milieu : l'espace latent."

- **1) VISUALISATION MINIMALE : $x \to f_\theta \to \hat{y}$ + PERTE**



- **2) OÙ VIT LA REPRÉSENTATION ? DANS LES COUCHES CACHÉES**

IDÉE CLÉ : LA REPRÉSENTATION EST « AU MILIEU »

Dans un réseau profond, on peut isoler une couche cachée comme représentation :

$$x \xrightarrow{f_{\theta_1}} z \xrightarrow{f_{\theta_2}} \hat{y}$$

Interprétation : z est un **vecteur** (ou un tenseur) qui résume l'entrée de façon utile.
Exemples :
- dans un **MLP** : z = activations d'une couche fully-connected,

- dans un **CNN** : z = feature maps (local) ou vecteur après global pooling (global),
- dans un **RNN** : z = état caché h_t (ou h_T),
- dans l'**attention** : z = représentation contextuelle (combinaison pondérée).

3) Ce qu'il faut retenir de ce schéma

EN UNE PHRASE

Que l'on parte de $x \to f_\theta \to \hat{y}$ ou de $x \to f_{\theta_1} \to z \to f_{\theta_2} \to \hat{y}$, c'est **exactement le même réseau** : on a juste choisi de nommer et d'isoler la couche intermédiaire z. C'est précisément **ce nom** — espace latent / représentation — qui va nous intéresser pour le reste du cours, bien plus que la valeur finale $\hat{y}$.

4) Mini-activité : préparer CNN/RNN

MINI-INTERACTION

Question : dans ce schéma, si je change la tâche (classification → génération), est-ce que z doit rester identique ?
Attendu : pas nécessairement. Changer de tâche peut modifier la représentation utile. Dans les **modèles génératifs à variables latentes**, on cherchera notamment un espace z compatible avec un mécanisme d'échantillonnage ; d'autres familles génératives procéderont autrement.

PHRASE DE LIEN (VERS CNN PUIS RNN)

Ce qui va vraiment nous intéresser aujourd'hui : *où* et *comment* chaque architecture fabrique z.
CNN : z vient de motifs **locaux** (feature maps). **RNN :** z vient d'une **mémoire séquentielle** (h_t).
Attention : z devient **contextuel**.

COMPLÉMENT NN — CE QU'IL FAUT SAVOIR POUR LA SUITE GÉNÉRATIVE

Les rappels précédents montrent déjà qu'un réseau construit une représentation. Pour la suite, retenez aussi quatre éléments :

- un neurone calcule typiquement une combinaison affine suivie d'une activation : $h = \sigma(Wx + b)$;
- la **fonction de perte** mesure l'écart entre ce que le modèle produit et l'objectif d'apprentissage ;
- la **rétropropagation** calcule les gradients nécessaires pour mettre à jour les paramètres θ ;
- les dimensions d'entrée et de sortie dépendent de la tâche : un réseau peut produire une étiquette, une représentation, les paramètres d'une distribution, ou directement une donnée générée.

Dans un générateur, l'entrée peut aussi être un bruit ou un vecteur latent z. Le rôle précis de ce latent dépendra de la famille de modèle étudiée.

A2 — CNN : du pixel aux features (localité, partage de poids, hiérarchie)

CNN — NOTIONS CLÉS À RETENIR

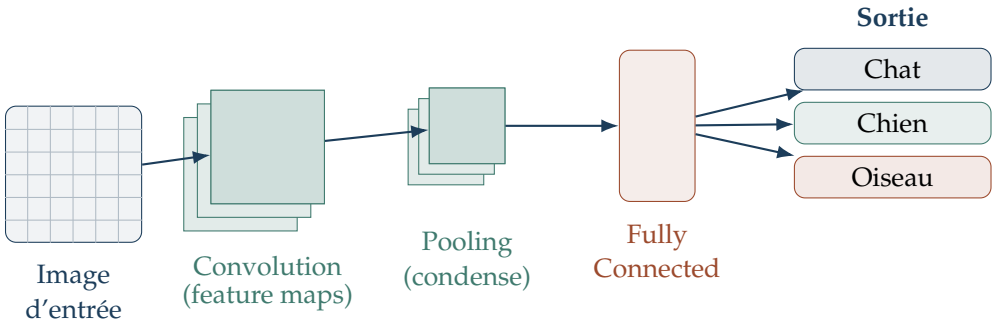
- **Localité :** un filtre 3×3 « voit » seulement un voisinage ⇒ détecte des motifs simples (bords, coins).
- **Partage de poids :** le *même* filtre est appliqué partout ⇒ un même motif peut être détecté à différentes positions avec les mêmes paramètres. La convolution est **équivariante à la translation** ; le pooling et d'autres agrégations peuvent ensuite apporter une certaine robustesse, voire une invariance approximative, aux déplacements.
- **Feature map :** la sortie d'un filtre est une **carte de caractéristiques** : chaque position indique «

le motif est présent ici ».

- **Hiérarchie (tendance observée) :** en empilant des couches, on observe *typiquement* une transformation progressive de l'espace :

$$\text{pixels} \rightarrow \text{bords} \rightarrow \text{textures} \rightarrow \text{parties} \rightarrow \text{concepts}$$

  Cette hiérarchie n'est pas garantie couche par couche, mais elle est largement confirmée par les méthodes de visualisation de features sur des CNN entraînés.
- **Où vit la représentation ?** dans les **feature maps** (représentation locale), puis dans un **vecteur** z après pooling/global pooling (représentation compacte = latent).



CNN : des pixels à la représentation globale z, jusqu'à une décision de classification.

A2 — CNN *(problème → échec → solution)*

Problème concret : une image est une grille de pixels, et les motifs utiles (bords, coins, textures) se répètent partout.
Échec (ancien outil) : un réseau dense « regarde tout d'un coup » ⇒ trop de paramètres, pas de notion naturelle de voisinage, faible invariance (un motif déplacé change tout).
Solution : CNN = filtres locaux + partage de poids + hiérarchie de features : on **change d'espace** (pixels → feature maps) puis on compresse vers un **latent** z.

MISE EN CONTEXTE — DU PIXEL AUX FEATURES

"Un CNN est conçu pour des données en grille, typiquement des images. Plutôt que de traiter chaque pixel isolément, il applique les mêmes filtres partout.
Du coup, il détecte d'abord des motifs simples (bords), puis, en empilant les couches, des motifs plus complexes (textures, parties d'objets, objets).
C'est déjà un changement d'espace : on passe de l'espace brut des pixels à un espace de **feature maps** où la similarité visuelle est mieux capturée."

- **1) LOCALITÉ : UNE FENÊTRE 3×3 SUFFIT POUR DÉTECTER UN MOTIF**

IDÉE (LOCALITÉ)

Un filtre 3×3 calcule une combinaison locale :

$$(\text{feature map})[i,j] = \sum_{u=-1}^{1}\sum_{v=-1}^{1} W[u,v]\; X[i+u,\, j+v]$$

Interprétation : le filtre parcourt l'image localement. Si un voisinage ressemble au motif appris par le filtre (par exemple un bord vertical), la valeur correspondante de la feature map devient élevée. Il n'est pas nécessaire, dans cette séance, de détailler davantage le calcul : l'idée essentielle est qu'un même détecteur local est appliqué partout dans l'image.

- **APERÇU CODE — CONVOLUTION, ACTIVATION ET POOLING**

Sans détailler une implémentation complète, le passage image → feature maps peut se lire ainsi :

```
features = conv(x) # filtres locaux partages
features = torch.relu(features)
z = pool(features) # representation plus compacte
```

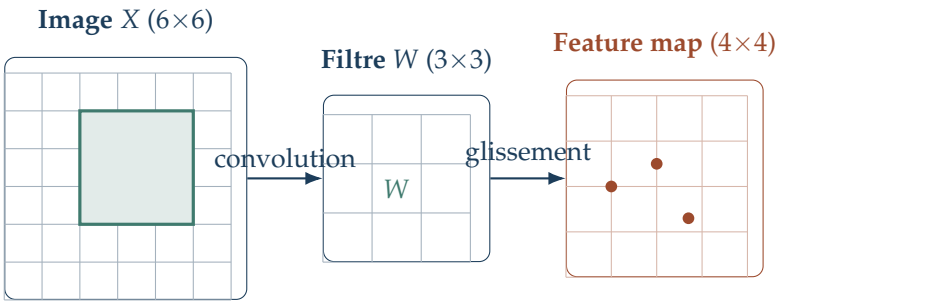
À observer : le code suit exactement l'idée du cours : appliquer des filtres locaux, produire des cartes de caractéristiques, puis les agréger vers une représentation plus compacte.

- **2) PARTAGE DE POIDS : LE MÊME MOTIF RECONNU PARTOUT**

IDÉE (PARTAGE DE POIDS)

Le même filtre W est utilisé à toutes les positions (i,j).
Conséquence : un bord détecté en haut à gauche ou en bas à droite active le *même* détecteur.
Gain : moins de paramètres qu'un dense + meilleure généralisation.

- **3) ILLUSTRATION VISUELLE : IMAGE 6×6 + FILTRE 3×3 → FEATURE MAP**



chaque case = **présence du motif** à cet endroit

- **4) HIÉRARCHIE ET LOCALISATION DE LA REPRÉSENTATION**

MESSAGE CLÉ

CNN = transformer progressivement l'espace des pixels en un espace de représentations plus abstraites.
Au fil des couches, la résolution spatiale peut diminuer tandis que les représentations tendent à devenir plus adaptées à la tâche :

$$\text{pixels} \to \text{bords} \to \text{textures} \to \text{parties/objets} \to z.$$

Cette progression est une intuition pédagogique utile : elle décrit une tendance fréquemment observée, mais elle n'impose pas qu'une couche donnée corresponde toujours exactement à un niveau sémantique précis.



feature maps = **représentations locales** z = **représentation globale**

OÙ VIT LA REPRÉSENTATION DANS UN CNN ?

Il n'existe pas un seul endroit obligatoire où « vit » la représentation :
- les **feature maps intermédiaires** représentent encore l'information spatiale localement ;
- après une opération d'agrégation (par exemple global pooling), on peut obtenir un vecteur z plus compact qui résume l'image globalement.

À retenir : le CNN ne se contente pas de classifier. Il construit progressivement des représentations qui transforment les pixels bruts en caractéristiques plus utiles pour la tâche.

LIMITE DU CNN (POUR LA GÉNÉRATION) *(problème → échec → solution)*

Le CNN construit une représentation z, mais uniquement **à partir de** x. Si on veut **créer** une image sans x en entrée, on ne sait pas quoi mettre dans z. → Le CNN utilisé ici comme encodeur construit z à partir de x. Pour générer sans disposer d'un x à encoder, il faut donc définir un mécanisme permettant de produire l'entrée du générateur.

On vient de voir que même un CNN puissant ne suffit pas à répondre à la question « d'où vient z ? ». Avant d'y revenir, complétons notre panorama des représentations en changeant de modalité : que devient cette même question quand les données sont du texte plutôt que des images ?

A3 — Texte : one-hot → embedding + RNN (séquence → état caché)

EMBEDDINGS & RNN — NOTIONS CLÉS À RETENIR

- **Problème du symbole :** un ID / one-hot n'a *pas* de géométrie (pas de notion de « proche »).
- **Embedding :** une **géométrie apprise** $e(w) \in \mathbb{R}^d$; selon l'objectif d'entraînement, certaines proximités peuvent refléter des similarités utiles.
- **RNN :** encode une **séquence** en une **mémoire compressée** : l'état caché h_t.
- **Message clé : RNN = encoder une séquence dans un état caché qui résume le passé.**
- **Où vit la représentation ?** dans (1) les embeddings des tokens et (2) les états cachés h_t (souvent h_T ou un pooling).

• A3.1 — One-hot : pourquoi ça échoue ?

A3.1 — ONE-HOT → EMBEDDING *(problème → échec → solution)*

Problème concret : pour le modèle, `chat` et `chien` devraient être plus proches que `chat` et `voiture`.
Échec (ancien outil) : one-hot : vecteurs orthogonaux (tous « à égale distance ») ⇒ **aucune notion de proximité sémantique**.
Solution : apprendre une table d'embedding : index ↦ vecteur dense $e(w)$, où la **distance** reflète une similarité apprise.

ILLUSTRATION (ONE-HOT)

One-hot : $\texttt{chat} = (0,0,\ldots,1,\ldots,0)$, $\texttt{chien} = (0,1,0,\ldots,0)$
Constat : $\langle \texttt{chat}, \texttt{chien} \rangle = 0$ (orthogonaux) ⇒ le modèle ne « voit » aucun lien.

- **ILLUSTRATION VISUELLE : TABLE D'EMBEDDING (EXEMPLE SIMPLE)**

Token	Vecteur embedding (dim 4 pour l'exemple)			
chat	0.2	-0.5	0.8	0.1
chien	0.3	-0.4	0.7	0.2
voiture	0.9	0.8	-0.1	-0.3

Exemple fictif à but pédagogique — ces valeurs ne sont pas issues d'un entraînement réel, elles servent uniquement à illustrer que des mots sémantiquement proches (chat/chien) ont des vecteurs numériquement proches.

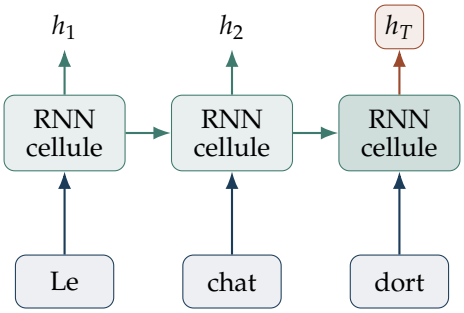
IDÉE À FAIRE DIRE AUX ÉTUDIANTS

Embedding = géométrie apprise.
Selon les données et l'objectif d'entraînement, des mots partageant des propriétés utiles peuvent obtenir des vecteurs proches (par exemple un cosinus élevé). Cette proximité n'est pas une garantie universelle de similarité sémantique.

- **A3.2 — RNN : L'ORDRE COMPTE (SÉQUENCE → ÉTAT CACHÉ)**

A3.2 — RNN (SÉQUENCE ET ÉTAT CACHÉ) *(problème → échec → solution)*

Problème concret : le sens d'une phrase dépend de l'ordre des mots (et du contexte temporel).
Échec (ancien outil) : « sac de mots » (ou moyenne d'embeddings) perd l'ordre ⇒ représentation insuffisante pour le sens.
Solution : un RNN lit la séquence pas à pas et maintient un **état caché** h_t qui joue le rôle de **mémoire compressée** de ce qu'il a vu.



Séquence lue token par token
l'état caché passe d'une cellule à l'autre

RNN : la même cellule est appliquée à chaque pas de temps ; l'état caché h_t résume ce qui a été lu jusqu'ici, jusqu'au résumé final h_T.

EXPLICATION PÉDAGOGIQUE — SÉQUENCE ET ÉTAT CACHÉ

"Pour le texte ou les séries temporelles, il ne suffit pas de regarder chaque élément isolément.
Un RNN lit la séquence pas à pas et maintient un état caché qui résume le passé.
Cet état, c'est une représentation latente : un vecteur qui résume la séquence."

FORME MINIMALE (SANS ENTRER DANS TOUS LES DÉTAILS)

À l'instant t :

$$h_t = f(h_{t-1}, e(x_t))$$

Interprétation : h_t contient « un résumé » de $x_1, \ldots, x_t$.

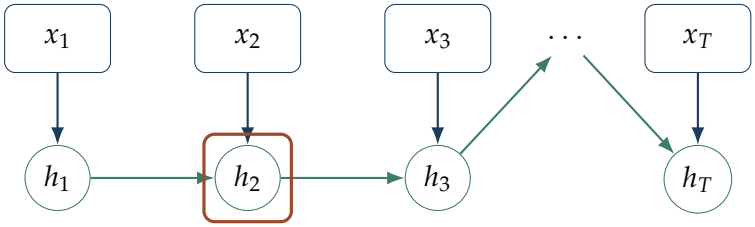
- **APERÇU CODE — MISE À JOUR DE L'ÉTAT CACHÉ**

La relation $h_t = f(h_{t-1}, e(x_t))$ correspond à une mise à jour répétée au fil de la séquence :

```
h = h0
for token in sequence:
    x_t = embedding(token)
    h = rnn_cell(x_t, h) # nouvel etat cache
```

À observer : à chaque étape, le nouvel état `h` dépend du token courant et de l'état précédent ; il accumule donc progressivement le contexte passé.

- **ILLUSTRATION VISUELLE RNN : SÉQUENCE → REPRÉSENTATION (ENTOURER h_t)**



h_t = représentation de la séquence $1 \ldots t$ h_T = représentation de la phrase

- **LIEN (1 PHRASE) : MÊME MOTIF QUE CNN**

PHRASE DE LIEN

On voit déjà un motif : que ce soit en image (CNN) ou en séquence (RNN), le réseau fabrique des **vecteurs** qui résument les données.
On se rapproche de l'idée centrale : **l'espace latent**.

LIMITES DU RNN (CE QUI NE MARCHE PAS BIEN) *(problème → échec → solution)*

1. **Contexte d'un RNN unidirectionnel standard** : lorsqu'il lit de gauche à droite, h_t n'utilise directement que le passé $x_{\leq t}$. Des architectures bidirectionnelles peuvent exploiter les deux directions. Le mot `boule` dans `boule de neige` ne peut pas utiliser `neige` pour se contextualiser.
2. **Mémoire limitée** : pour les longues phrases, l'information du début se dilue au fil des étapes (on parle de *vanishing gradient* : le signal d'erreur utilisé pour l'apprentissage devient de plus en plus faible à mesure qu'il remonte vers les premières positions de la séquence, ce qui rend leur mise à jour de plus en plus difficile).
3. **Séquentiel obligatoire** : on ne peut pas paralléliser la lecture d'une phrase → lent à l'entraînement.

- **MINI-INTERACTION**

MINI-INTERACTION

Question : pourquoi one-hot est « catastrophique » pour représenter la similarité ?
Attendu : vecteurs orthogonaux ⇒ aucune notion de proximité (pas de géométrie).

A4 — Synthèse A : les représentations internes comme langage commun

A4 — SYNTHÈSE A *(problème → échec → solution)*

Problème concret : images et séquences ont des formats différents.
Échec : l'espace brut ne porte pas le sens.
Solution : les architectures apprennent des **représentations internes** dans lesquelles certaines relations pertinentes pour la tâche peuvent devenir plus faciles à exprimer.

Architecture	Où est la représentation ?	Nature
CNN	feature maps profondes + global pooling $\rightarrow z$	hiérarchie spatiale
RNN	états cachés h_t ; souvent h_T	séquentielle
Embedding	vecteur token $e(w)$	discret $\rightarrow$ continu

DÉFINITION (SIMPLE MAIS SOLIDE)

Espace latent / espace de représentation : espace interne appris par le modèle, parfois de dimension réduite, dans lequel sont encodées certaines caractéristiques des données.
Deux points proches **peuvent** correspondre à des exemples similaires selon la représentation apprise et l'objectif d'entraînement ; cette propriété n'est pas garantie universellement.

À ce stade du cours, la lettre z désigne simplement une **représentation interne**. Plus tard, notamment avec les modèles à variables latentes, le rôle de z deviendra plus précis : z sera alors une **variable aléatoire** à laquelle on associera une distribution $p(z)$.

COMPLÉMENT — TROIS ESPACES À NE PAS CONFONDRE

Espace de caractéristiques : activations internes apprises pour rendre une tâche plus simple.
Embedding supervisé : représentation dont la géométrie est influencée par une tâche et des étiquettes.
Espace latent probabiliste : espace associé à une variable aléatoire Z et à une distribution, par exemple $z \sim p(z)$.
Ces notions peuvent se recouvrir selon l'architecture, mais elles ne sont pas synonymes.

EXEMPLE VISUEL À IMAGINER : MNIST

Supposons qu'un réseau transforme chaque image de chiffre en un point $z \in \mathbb{R}^2$. Après projection, les « 1 » peuvent former un groupe et les « 7 » un autre. Cela montre que la représentation a organisé certaines similarités utiles. Mais deux points proches ne sont pas nécessairement sémantiquement proches dans tout modèle : la géométrie dépend de l'objectif d'apprentissage.

INTERPOLATION ET LIMITE DE L'AUTOENCODEUR CLASSIQUE

Entre deux codes z_1 et z_2, on peut explorer un point intermédiaire

$$z_\alpha = (1-\alpha)z_1 + \alpha z_2, \qquad 0 \leq \alpha \leq 1.$$

Décoder ces points peut produire une transition intéressante si l'espace latent est bien structuré. Cependant, un autoencodeur classique ne garantit pas qu'entre deux codes observés, toutes les régions soient plausibles ou bien couvertes. Cette limite préparera directement l'étude des VAE.

TRANSITION VERS LA PARTIE B

On sait maintenant **où** se trouve la représentation.
Question suivante : est-elle **fixe** (un mot = un seul vecteur) ou **contextuelle** (le vecteur dépend de la phrase) ?

3 Transformers & Attention : représentations contextuelles

3.1 Self-Attention : du token fixe au token contextualisé

IDÉE CLÉ

- Un même token n'a pas toujours le même sens : sa représentation doit dépendre du **contexte**.
- La self-attention construit, pour chaque position t, un vecteur contextuel h_t comme **combinaison pondérée** des valeurs associées aux positions autorisées de la séquence.

- **DE X VERS Q, K, V (PROJECTIONS APPRISES)**

Soit $X \in \mathbb{R}^{T \times d}$ la matrice des embeddings (T tokens, dimension d). On calcule :

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

où W_Q, W_K, W_V sont des paramètres apprenants. On peut les lire intuitivement comme : **Q** = ce que le token recherche, **K** = ce que chaque token peut faire correspondre, et **V** = l'information effectivement transmise.

- **SCALED DOT-PRODUCT ATTENTION (FORMULE CANONIQUE)**

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)\, V$$

Rappel : le *softmax* transforme les scores d'attention en poids positifs dont la somme vaut 1. Ces poids indiquent donc quelle importance chaque position reçoit.

- $QK^\top$: matrice de similarité (qui regarde qui ?)
- $\frac{1}{\sqrt{d_k}}$: stabilise l'amplitude des scores quand d_k grand (sinon softmax sature).
- M : masque utilisé notamment pour la self-attention causale du décodeur (**0** si une position est autorisée, $-\infty$ si elle est interdite — en pratique, un très grand nombre négatif fini).
- Multiplication par V : agrégation de l'information (valeurs) avec ces poids.

- **APERÇU CODE — DE LA FORMULE D'ATTENTION AUX OPÉRATIONS MATRICIELLES**

La formule d'attention se traduit presque directement en quelques opérations :

```
scores = Q @ K.transpose(-2, -1)
scores = scores / math.sqrt(d_k)
weights = torch.softmax(scores + mask, dim=-1)
H = weights @ V
```

À observer : on calcule d'abord les scores de similarité, on les normalise en poids d'attention, puis on combine les valeurs V pour obtenir une représentation contextuelle H.

MINI-EXEMPLE : COMPRENDRE LE MASQUE CAUSAL

Considérons trois tokens (w_1, w_2, w_3). Lorsque le modèle calcule la représentation de la position 2, il peut utiliser w_1 et w_2, mais il ne doit pas regarder w_3 si celui-ci appartient au futur. Le masque attribue donc une valeur très négative au score correspondant à cette position future ; après le softmax, son poids devient pratiquement nul.

Idée à retenir : le masque causal ne change pas le mécanisme d'attention ; il indique simplement **quelles positions sont autorisées à être consultées**.

MINI-INTERACTION

Question : dans $QK^\top$, si le score entre le token « chat » (en position requête) et le token « miaule » (en position clé) est très élevé après softmax, qu'est-ce que cela signifie concrètement pour le calcul de h_{chat} ?
Attendu : le vecteur h_{chat} (la sortie de l'attention pour « chat ») accordera un poids important à la valeur V associée à « miaule » dans sa somme pondérée — « chat » « regarde » fortement « miaule » pour se contextualiser.

- **TROIS MÉCANISMES D'ATTENTION DU TRANSFORMER ENCODEUR–DÉCODEUR CLASSIQUE**

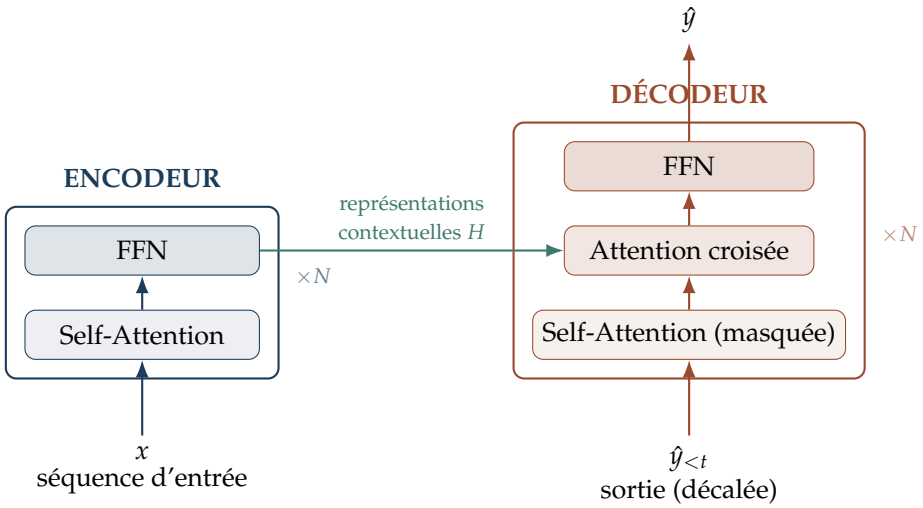
POURQUOI TROIS MÉCANISMES D'ATTENTION DANS L'ARCHITECTURE CLASSIQUE ? *(problème → échec → solution)*

Self-attention de l'encodeur : chaque position peut exploiter le contexte de la séquence d'entrée.
Self-attention masquée du décodeur : génération autorégressive sans accès aux tokens futurs.
Attention croisée encodeur–décodeur : le décodeur **interroge** les représentations produites par l'encodeur.

3.2 Architecture Transformer : blocs empilés + résidus + normalisation

BLOC TRANSFORMER (NOYAU)

- **(1) Multi-Head Attention** : contextualise les tokens
- **(2) FFN (MLP position-wise)** : transforme non-linéairement chaque position
- **(3) Add & Norm** : connexions résiduelles + normalisation (stabilité)

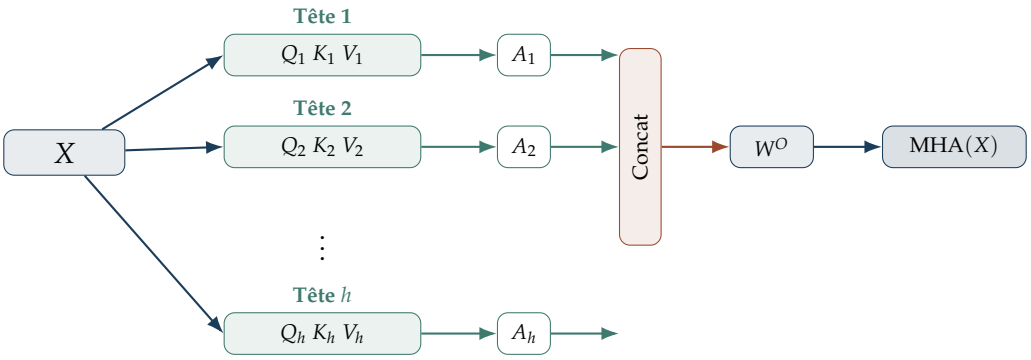


Architecture simplifiée d'un Transformer encodeur–décodeur : l'encodeur empile N blocs self-attention + FFN pour construire des représentations contextuelles H ; le décodeur combine self-attention masquée, attention croisée (sur H) et FFN pour générer la sortie.

PASSAGE D'UNE TÊTE À PLUSIEURS TÊTES

Une tête d'attention applique le mécanisme vu précédemment avec son propre jeu de projections Q, K et V. La **Multi-Head Attention** répète simplement ce même mécanisme plusieurs fois avec des projections apprises différentes. Les sorties des têtes sont ensuite concaténées puis reprojetées.

L'intérêt est que plusieurs relations contextuelles peuvent être considérées en parallèle. Certaines têtes peuvent se spécialiser, mais cette spécialisation n'est pas imposée.



Multi-Head Attention : X est projeté en h sous-espaces (Q_i, K_i, V_i) ; chaque tête calcule sa propre attention A_i, puis les sorties sont concaténées et reprojetées par W^O.

DEUX COMPLÉMENTS D'ARCHITECTURE À CONNAÎTRE

Résidus et LayerNorm. Les connexions résiduelles facilitent l'optimisation des réseaux profonds, tandis que la Layer Normalization aide à stabiliser les activations. Pour cette semaine, il suffit de comprendre leur **rôle général** ; leur détail algorithmique n'est pas nécessaire.

Encodage positionnel. La self-attention seule ne représente pas explicitement l'ordre des tokens. On ajoute donc à l'embedding du token une information de position :

$$x_t = e(w_t) + p_t.$$

Ici, p_t indique la position t dans la séquence. Il peut être **appris** ou construit à l'aide de fonctions **sinusoïdales**. L'idée essentielle est simplement que le modèle doit savoir si un token apparaît en première, deuxième ou dixième position.

FORMULE MULTI-HEAD ATTENTION À RECONNAÎTRE

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V), \qquad \text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O.$$

Chaque tête applique le même principe d'attention avec des projections différentes ; leurs résultats sont ensuite regroupés. Il n'est pas nécessaire de mémoriser cette formule dès maintenant, mais il faut savoir la relier au schéma précédent.

3.3 Aperçu : entraînement et génération d'un décodeur causal

POURQUOI DISTINGUER ENTRAÎNEMENT ET GÉNÉRATION ?

Dans un décodeur autoregressif, le modèle apprend à prédire le token suivant à partir des tokens précédents. Pendant l'entraînement, la vraie séquence est disponible ; pendant la génération, elle ne l'est plus.

Pour une séquence cible $(y_1^\star, \ldots, y_T^\star)$, une perte typique s'écrit :

$$\mathcal{L} = -\sum_{t=1}^{T} \log p(y_t^\star \mid y_{<t}^\star, x).$$

À chaque position t, le modèle reçoit le contexte précédent $y_{<t}^\star$ et doit attribuer une probabilité élevée au vrai token suivant $y_t^\star$. La somme additionne cette pénalité sur toutes les positions de la séquence.

Pendant l'**entraînement**, on utilise généralement les vrais tokens précédents comme contexte : c'est le **teacher forcing**. Pendant la **génération**, le vrai token suivant n'est pas disponible ; le modèle produit un token, puis réutilise ses propres sorties pour continuer.

CE QUI SERA APPROFONDI PLUS TARD

Cette mécanique est un premier aperçu de la génération **auto-régressive**. Nous la reprendrons en détail en **S5**, avec la règle de chaîne, PixelCNN/PixelRNN et la génération séquentielle. Ici, retenez seulement pourquoi le masque causal et la prédiction du prochain élément sont liés.

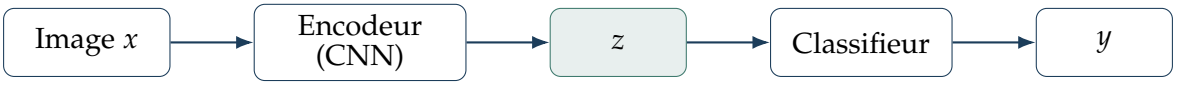
Nous avons maintenant vu comment une représentation contextuelle peut aussi servir à produire une sortie séquentielle. Revenons à la question centrale de cette semaine : quelle différence y a-t-il entre reconstruire une observation existante et générer une nouvelle observation plausible ?

4 Partie C : Génération vs Reconstruction

C1 — Paradigme classique : reconnaissance

C1 — RECONNAISSANCE *(problème → échec → solution)*

Problème concret : prédire une étiquette y à partir d'une entrée x.
Outil : $x \to z \to y$ (encodeur + classifieur).
Limite : z est un sous-produit ; on apprend surtout $p(y|x)$, pas comment **produire** des x.



pas de mécanisme de génération de x

C2 — Génération : produire une nouvelle observation

C2 — GÉNÉRATION *(problème → échec → solution)*

Problème concret : produire une nouvelle observation plausible sans disposer de cette observation en entrée.
Échec : dans le paradigme précédent, la représentation interne est construite à partir de x ; cela ne suffit donc pas, à lui seul, à définir un mécanisme de génération.
Solution : apprendre un **mécanisme génératif**. Dans une famille importante de modèles à variables latentes, on définit un prior $p(z)$ puis on génère selon

$$z \sim p(z), \qquad x \sim p_\theta(x \mid z).$$

D'autres familles génératives procéderont autrement ; nous les classifierons en S2.

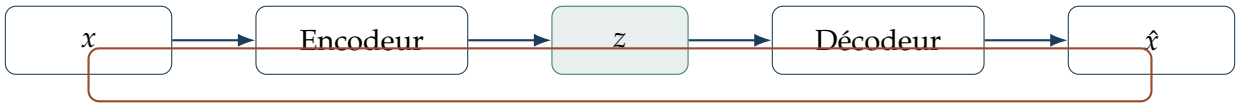
exemple latent : $z \sim p(z)$



C3 — Expérience mentale : l'autoencodeur

C3 — AUTOENCODEUR *(problème → échec → solution)*

Problème concret : compresser puis reconstruire : $x \to z \to \hat{x}$.
Solution : minimiser $\|x - \hat{x}\|^2$.
Limite : l'autoencodeur classique n'impose pas de **prior explicite** sur les codes z. Rien ne garantit donc que les régions entre les codes observés soient régulièrement occupées ou facilement échantillonnables.



perte : $\|x - \hat{x}\|^2$

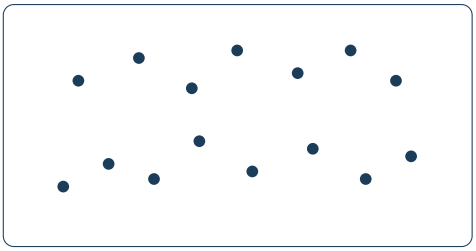
- **ILLUSTRATION : ESPACE LATENT ET ÉCHANTILLONNAGE (INTUITION)**

IDÉE CLÉ : POURQUOI L'AE CLASSIQUE NE SUFFIT PAS POUR GÉNÉRER ?

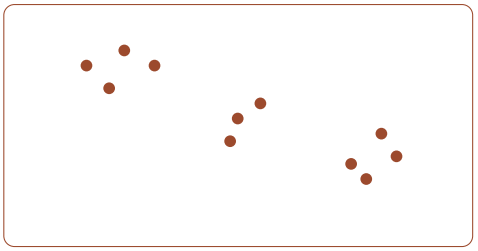
Dans un autoencodeur classique, l'encodeur apprend des codes z utiles pour **reconstruire** les observations, mais il n'impose pas de prior explicite qui organise ces codes pour l'échantillonnage. Certaines régions de l'espace latent peuvent donc être peu couvertes.

Si l'on choisit arbitrairement un point dans une telle région et qu'on le décode, la sortie peut être peu plausible. Les modèles génératifs à variables latentes traiteront précisément cette difficulté ; elle sera approfondie dans les semaines consacrées aux autoencodeurs et aux VAE.

Espace latent régularisé facilement échantillonnable



AE classique : codes latents sans prior explicite



Un point choisi dans une région peu couverte peut produire une sortie peu plausible.

C4 — Reconstruction et génération : deux objectifs différents

Reconstruction	Génération
Une observation d'entrée est disponible	On cherche à produire une nouvelle observation
La sortie est contrainte par cette entrée	La sortie est obtenue par un mécanisme de génération
Objectif principal : retrouver/préserver l'information pertinente	Objectif principal : produire un échantillon plausible
Exemple AE : $x \to z \to \hat{x}$	Exemple latent : $z \sim p(z)$, puis $x \sim p_\theta(x \mid z)$

SAUT CONCEPTUEL (EN GROS) *(problème → échec → solution)*

Générer signifie disposer d'un mécanisme capable de produire de **nouveaux échantillons plausibles**.
Dans les modèles génératifs à variables latentes, cela peut prendre la forme $z \sim p(z)$ puis $x \sim p_\theta(x \mid z)$. Dans un AE classique, la distribution des codes encode(x) n'est pas explicitement régularisée pour permettre un tel échantillonnage.

MINI-INTERACTION

Question : un modèle qui prend une photo floue et la rend nette — est-ce nécessairement de la **reconstruction** ou de la **génération** ?
Attendu : dans le cadre simplifié de cette séance, on peut la voir comme une **reconstruction/restauration conditionnée par une entrée**, puisque la sortie est guidée par la photo floue. Plus généralement, des modèles génératifs conditionnels peuvent aussi réaliser ce type de tâche ; la frontière dépend donc du modèle et de l'objectif probabiliste.

5 Complément : échantillonnage et plausibilité

DEUX OPÉRATIONS À NE PAS CONFONDRE

Évaluer une donnée : on part d'une observation déjà disponible x et l'on cherche à savoir si elle est compatible avec ce que le modèle a appris. Lorsque le modèle fournit une densité, cette idée peut être reliée à $p_\theta(x)$.
Échantillonner : on utilise le mécanisme appris pour produire une nouvelle réalisation qui n'était pas fournie en entrée.

MINI-EXERCICE

Indiquez l'opération dominante :

1. compresser une photo puis reconstruire cette même photo ;
2. examiner si une observation est plausible selon le modèle ;
3. tirer un latent aléatoire puis produire une nouvelle image.

Réponses attendues : reconstruction ; évaluation de plausibilité ; génération par échantillonnage.

6 Synthèse de la semaine 1

SYNTHÈSE

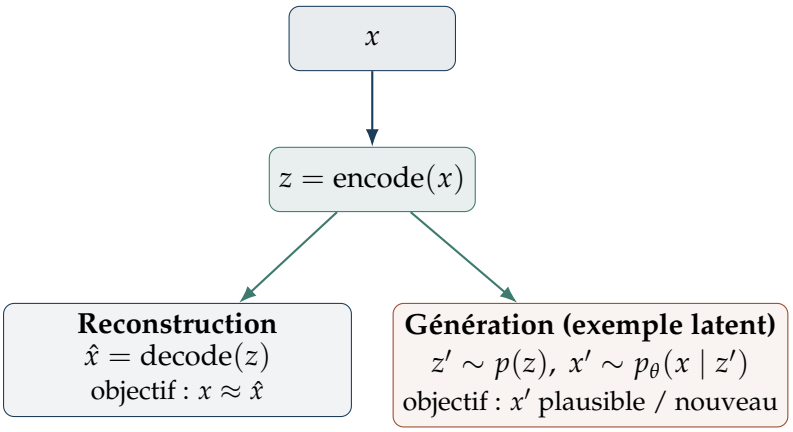
"Aujourd'hui, on a appris à **construire** des représentations : CNN (hiérarchie), RNN (séquence), Attention (contexte).
Mais générer exige maintenant de comprendre comment passer d'une représentation utile à un mécanisme capable de produire de nouvelles observations plausibles. Dans certains modèles à variables latentes, cela passera par un espace latent échantillonnable ; d'autres familles procéderont autrement.
Au prochain cours (S2), nous changerons de point de vue : nous étudierons formellement **quelle distribution un modèle cherche à apprendre**."

RÉCAPITULATIF

On a vu dans ce cours de la semaine 1, les architectures étudiées apprennent des **représentations internes** adaptées à la structure des données : CNN pour l'espace visuel, RNN pour les séquences, attention pour les représentations contextuelles, et *embeddings* pour associer aux symboles une géométrie apprise.

La grande question qui nous accompagne maintenant, c'est : **comment passer de représentations utiles à un mécanisme capable de *générer* de nouvelles données plausibles**, et pas seulement de reconnaître ou de reconstruire une entrée ?

À venir (S2) : nous utiliserons les notions de donnée, distribution et échantillon introduites ici pour comparer formellement les différents objectifs probabilistes d'apprentissage. La taxonomie des grandes familles génératives sera alors introduite, sans la développer prématurément en S1.



on a x → on reconstruit pas d'observation cible en entrée → on génère

Note : la reconstruction part d'une observation x et cherche à la retrouver ; la génération cherche à produire une nouvelle observation. Le chemin $z \sim p(z)$ puis $x \sim p_\theta(x \mid z)$ illustre ici le cas des modèles génératifs à variables latentes.

AUTO-VÉRIFICATION — S1

Avant de passer en S2, vérifiez que vous pouvez répondre sans regarder les pages précédentes :

1. Quelle différence existe entre X, x et x_i ?
2. Que représentent $p_{\text{data}}(x)$ et $p_\theta(x)$ dans l'intuition introduite en S1 ?
3. Quelle différence existe entre une probabilité discrète et une densité continue ?
4. Que signifie $z = f_\theta(x)$ dans un réseau ?

5. Où se trouve la représentation dans un CNN, un RNN et un mécanisme d'attention ?
6. Pourquoi deux points proches dans un espace latent ne sont-ils pas nécessairement sémantiquement proches ?
7. Quelle différence existe entre reconstruire une observation et générer une nouvelle observation ?
8. Que signifie échantillonner depuis un modèle ?
9. Pourquoi une bonne reconstruction ne garantit-elle pas automatiquement une bonne génération ?